

- Which shared Trusted Root to select
- Which Fabric ID to use for validation of initiator and responder operational certificates
- Which Node ID is targeted in the given Fabric

This serves several purposes:

1. It requires an initiator to have knowledge of both the [IPK](#) and one of the full identities of the responder Node before it forces the responder node to generate a costly [Sigma2](#) message
 - a. Note that the replay of previously recorded initiator messages is possible, and therefore a Node MAY choose to keep memory of some prior destination identifiers that were successfully processed which it would later reject if seen again, for additional replay protection
2. It ensures that there is no ambiguity on the responder as to which Fabric was selected for communication
3. It hides which Fabric was chosen by the initiator

A destination identifier is generated by:

1. Concatenating the following octet strings for subsequent usage as a [destinationMessage](#):
 - [initiatorRandom](#): The value of [initiatorRandom](#) that will be used in the same message as the Destination Identifier
 - [rootPublicKey](#): The public key of the root of trust of the desired fabric, from the [ec-pub-key](#) field of the [Matter Certificate](#) of that root, as an uncompressed elliptic curve point as defined in section 2.3.3 of [SEC 1](#)
 - [fabricId](#): The Fabric ID of the destination, matching the [matter-fabric-id](#) field of the [Matter Certificate](#) of the desired destination's [NOC](#), and encoding the 64-bit scalar as a little-endian byte order octet string
 - [nodeId](#): The Node ID of the destination, matching the [matter-node-id](#) field of the [Matter Certificate](#) of the desired destination's [NOC](#), and encoding the 64-bit scalar as a little-endian byte order octet string
2. Obtaining the appropriate [Identity Protection Key \(IPK\) Operational Group Key](#) for the associated [Fabric](#) under [Group Key Set index](#) 0 within the [Group Key Management Cluster](#).
3. Computing an identifier [destinationIdentifier](#) of length [CRYPTO_HASH_LEN_BYTES](#) using [Crypto_HMAC\(\)](#) with the [IPK](#) as the [key](#) and [destinationMessage](#) as the [message](#)

The above steps can be summarized as:

```
destinationMessage = initiatorRandom || rootPublicKey || fabricId || nodeId
destinationIdentifier = Crypto_HMAC(key=IPK, message=destinationMessage)
```

For example, given the following:

- Root public key for the common Fabric, in uncompressed elliptical curve point form: